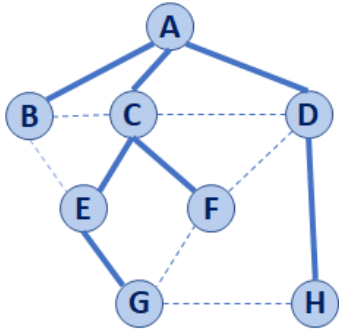


BFS Graph Traversal



d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G

Pseudocode for BFS

```

1  BFS(G):
2    Input: Graph, G
3    Output: A labeling of the edges on
4           G as discovery and cross edges
5
6    foreach (Vertex v : G.vertices()):
7      setLabel(v, UNEXPLORED)
8    foreach (Edge e : G.edges()):
9      setLabel(e, UNEXPLORED)
10   foreach (Vertex v : G.vertices()):
11     if getLabel(v) == UNEXPLORED:
12       BFS(G, v)
13
14  BFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23          setLabel(v, w, DISCOVERY)
24          setLabel(w, VISITED)
25          q.enqueue(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27          setLabel(v, w, CROSS)

```

BFS Observations

- Does our implementation handle disjoint graphs? How?
 - How can we modify our code to count components?
- Can our implementation detect a cycle? How?
 - How can we modify our code to store update a private member variable `cycleDetected_`?
- What is the running time of our algorithm?
- What is the shortest path between **A** and **H**?
- What is the shortest path between **E** and **H**?
 - What does that tell us about BFS?
- What does a cross edge tell us about its endpoints?
- What structure is made from discovery edges in **G**?

Big Ideas: Utility of a BFS Traversal

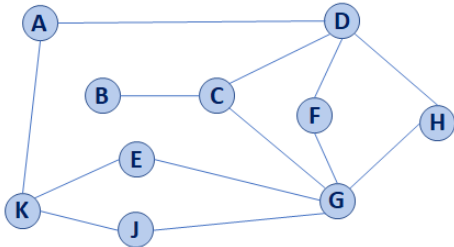
Obs. 1: Traversals can be used to count components.

Obs. 2: Traversals can be used to detect cycles.

Obs. 3: In BFS, **d** provides the shortest distance to every vertex.

Obs. 4: In BFS, the endpoints of a cross edge never differ in distance, **d**, by more than 1: $|\mathbf{d(u)} - \mathbf{d(v)}| = 1$

Depth First Search – A Modification to BFS



Two types of edges: 1.

2.

Running Time of DFS:

Labeling:

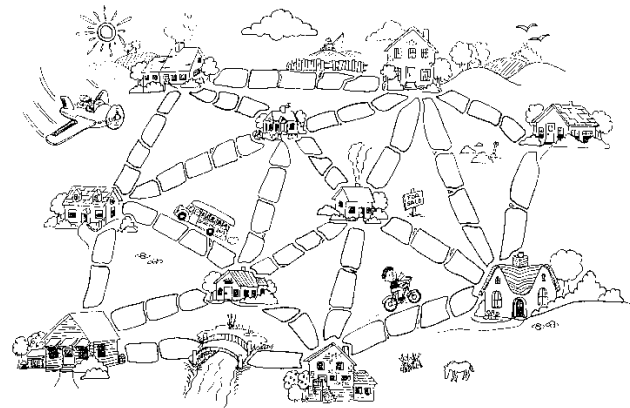
- Vertex:
- Edge:

Queries:

- Vertex:
- Edge:

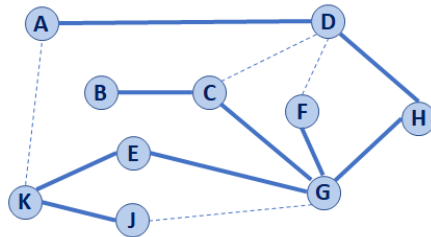
Pseudocode for DFS

```
1 BFS(G):
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
13
14 BFS(G, v):
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```



[“The Muddy City” by CS Unplugged](#), Creative Commons BY-NC-SA 4.0

Depth First Search (DFS) Traversal



DFS Traversal Starting at A, moving clockwise around edges.

```

1 DFS(G):
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and back edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
13
14  DFS(G, v):
15      setLabel(v, VISITED)
16
17      foreach (Vertex w : G.adjacent(v)):
18          if getLabel(w) == UNEXPLORED:
19              setLabel(v, w, DISCOVERY)
20              q.enqueue(w)
21          elseif getLabel(v, w) == UNEXPLORED:
22              setLabel(v, w, BACK)

```

Running Time of DFS:

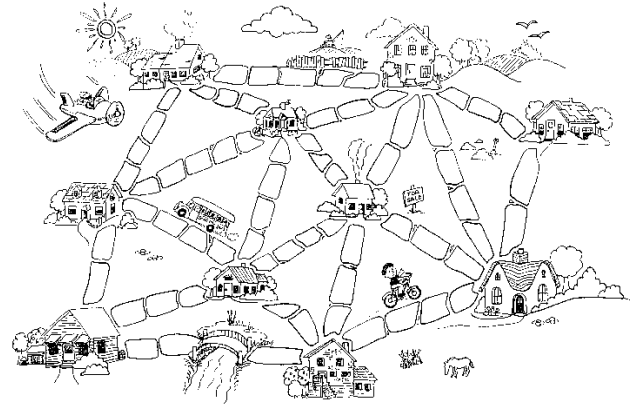
Labeling:

- Vertex:
- Edge:

Queries:

- Vertex:
- Edge:

Spanning Trees



"The Muddy City" by CS Unplugged, Creative Commons BY-NC-SA 4.0

Q: What road should we build first?

Q: What strategy should we use to build the next road?

A **Spanning Tree** on a connected graph G is a subgraph, G' , such that:

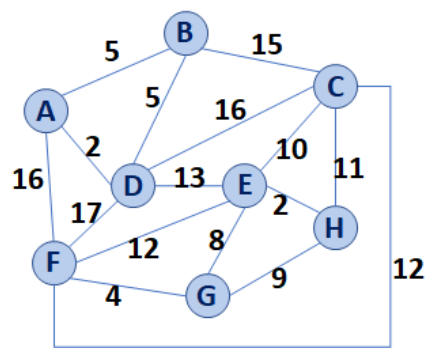
1. Every vertex in G is in G' and
2. G' is connected with the minimum number of edges

This construction will always create a new graph that is a tree (connected, acyclic graph) that spans G .

A **Minimum Spanning Tree** is a spanning tree with the minimal total edge weights among all spanning trees.

- Every edge must have a weight
 - The weights are unconstrained, except they must be additive (*eg: can be negative, can be non-integers*)
- Output of a MST algorithm produces G' :
 - G' is a spanning graph of G
 - G' is a tree
 - G' has a minimal total weight among all spanning trees

Kruskal’s Algorithm



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal’s Running Time Analysis

We have multiple choices on which underlying data structure to use to build the Priority Queue used in Kruskal’s Algorithm:

Priority Queue Implementations:	Heap	Sorted Array
Building : 7-9		
Each removeMin : 13		

Based on our algorithm choice:

Priority Queue Implementation:	Total Running Time
Heap	
Sorted Array	

Pseudocode for Kruskal’s MST Algorithm	
1	KruskalMST(G) :
2	DisjointSets forest
3	foreach (Vertex v : G) :
4	forest.makeSet(v)
5	
6	PriorityQueue Q // min edge weight
7	foreach (Edge e : G) :
8	Q.insert(e)
9	
10	Graph T = (V, {})
11	
12	while T.edges() < n-1:
13	Edge (u, v) = Q.removeMin()
14	if forest.find(u) != forest.find(v) :
15	T.addEdge(u, v)
16	forest.union(forest.find(u) ,
17	forest.find(v))
18	
19	return T

Kruskal's Algorithm

Pseudocode for Kruskal's MST Algorithm

```

1  KruskalMST(G):
2    DisjointSets forest
3    foreach (Vertex v : G):
4      forest.makeSet(v)
5
6    PriorityQueue Q    // min edge weight
7    foreach (Edge e : G):
8      Q.insert(e)
9
10   Graph T = (V, {})
11
12   while |T.edges()| < n-1:
13     Edge (u, v) = Q.removeMin()
14     if forest.find(u) != forest.find(v):
15       T.addEdge(u, v)
16       forest.union( forest.find(u),
17                    forest.find(v) )
18
19   return T

```

Based on our algorithm choice:

Priority Queue Implementation:	Total Running Time
Heap	$O(n + m) + O(m \lg(n))$
Sorted Array	$O(n + m \lg(n)) + O(m)$

Why would we prefer a Heap?

Why would be prefer a Sorted Array?

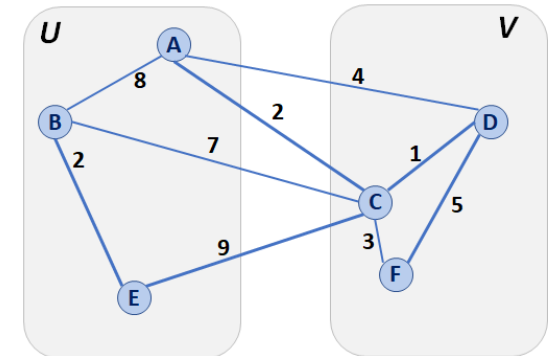
Partition Property

Consider an arbitrary partition of the vertices on G into two subsets U and V .

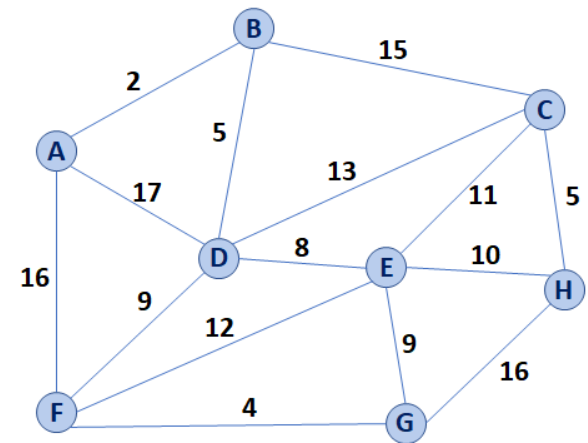
Let e be an edge of minimum weight across the partition.

Then e is part of some minimum spanning tree.

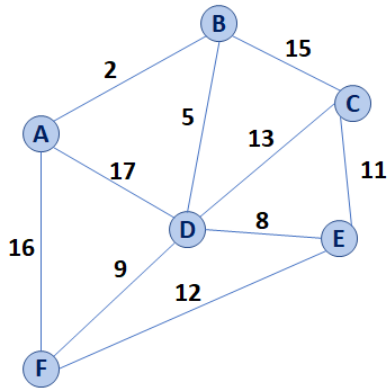
Proof in CS 374!



Partition Property Algorithm



Prim's Algorithm



Pseudocode for Prim's MST Algorithm

```

1  PrimMST(G, s):
2    Input: G, Graph;
3          s, vertex in G, starting vertex of algorithm
4    Output: T, a minimum spanning tree (MST) of G
5
6    foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9    d[s] = 0
10
11   PriorityQueue Q  // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T

```

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Running Time of MST Algorithms

Kruskal's Algorithm:

Prim's Algorithm:

Q: What must be true about the connectivity of a graph when running an MST algorithm?

...what does this imply about the relationship between **n** and **m**?

CS 225 – Things To Be Doing:

1. MP7 extra due Monday, May 18
2. MP7 final due Monday, May 25
3. lab_graphs due Sunday, May 17